
From traditional SDLC to AI-native

Issues we see today

Belief

Nobody trusts the output, and everybody expects miracles. Those are the same mistake made in opposite directions.

Capability

No training, and processes built for a world where writing code was the expensive part.

Foundation

Product knowledge lives in people's heads. AI workflows are fragmented, each as good as whoever built it.

Tooling

Nothing is unified, there is no common way of working, and the average developer has too little AI to learn on.

The hardest constraint is not technical

"We have always done it this way"

Answered by input from outside the team. Someone who has already made the move, in the room, not a memo about it.

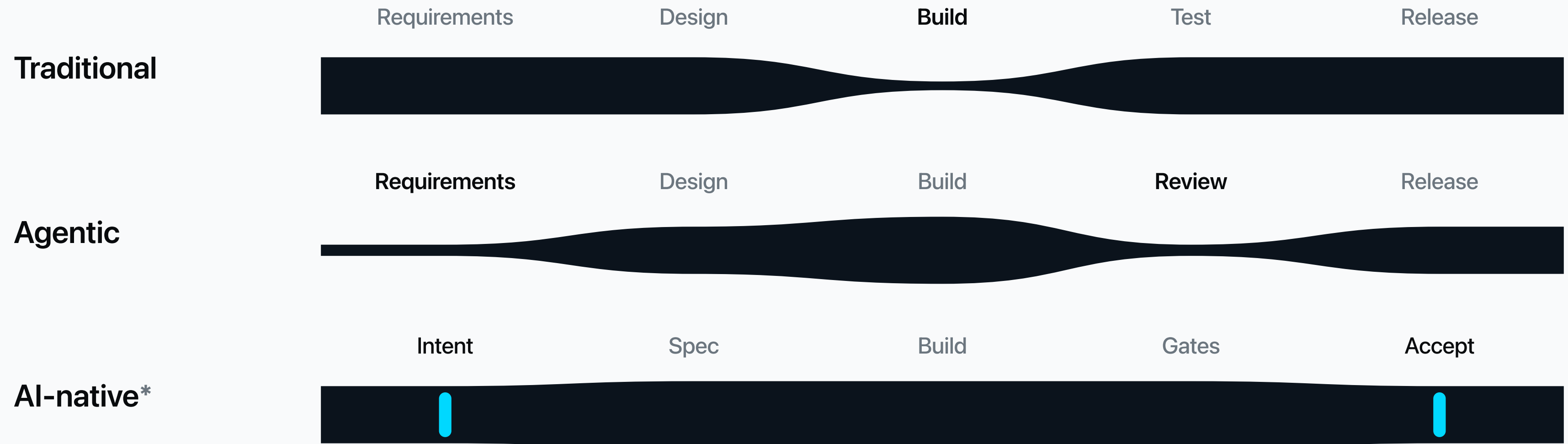
"I do not trust it, my code is better"

Answered by demonstration. A real repository, a real change, measured against what the team would have shipped. Evidence beats argument.

"It only creates slop"

Answered by gates. Slop is what unchecked output looks like, and there is no verification and validation process in front of it today.

The bottleneck moves. It does not disappear.



Based on Anthropic's AI-native SDLC playbook.

* The ideal shape. There will still be hiccups in the chain.

You cannot skip ahead. Maturity and trust are what the rest is built on.

The middle stage is where the people change

It is the stage where a team can learn the new way of working while the stakes are still low.

Learning in phases

Nobody goes from writing every line to reviewing an agent's work overnight. This is the step in between.

Trusting the output

Trust comes from working next to it on real code, not from a policy that says it is allowed.

Living with gates

Once an agent writes the volume, a gate stops being bureaucracy and becomes what makes the volume safe.

What we propose

Quality gates

Quality becomes visible and automatic in CI/CD.
Nobody reviews volume by hand.

Platform

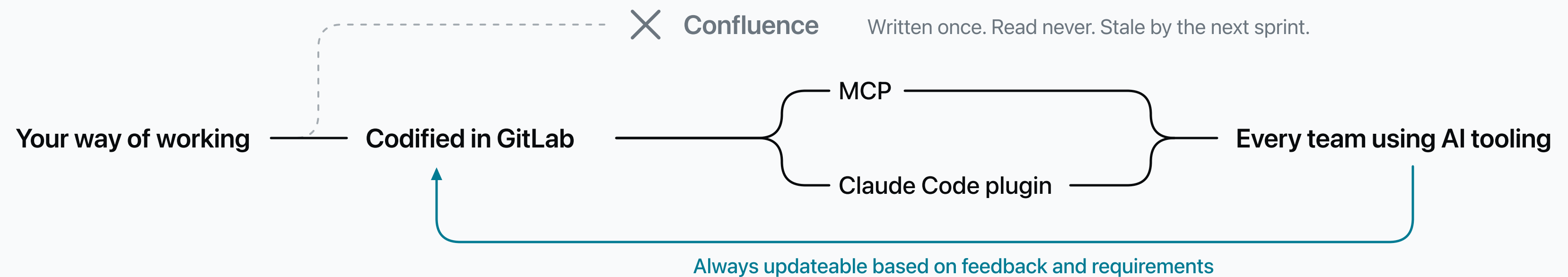
One way of working, shipped to every team.
Your standards carried by the tooling instead of
remembered by people.

Training

Teams learn to work with agents rather than
around them.

Access without gates buys you more code and less confidence in it.

Platform engineering



The agent needs somewhere to work



Quality gates only help if the agent can run them itself. Otherwise they are a queue with a person at the front.

"Do you know of MCP?"

Asked in earnest, by people already using AI. Alongside it: the agent used as a search box, and pointed at small routine tasks instead of whole workflows.

The gap is not willingness. People do not know how to use it well, and training closes that.

What we need from you

Priority

Platform engineering as the primary track, Ponto and antifraud as secondary.
Decided once, so it is not reopened per team.

Resources

The teams that move need the tooling to move with: more Izzy credits, or an economic model built for daily use, such as a Claude subscription.

People

Evangelists placed inside the teams, with coaching and changing minds as the job, not as something on top of delivery.

Feedback

A standing loop back to us, so we steer on what the teams actually run into instead of on what we assumed.

All four rest on a mandate that cuts across teams. Without it, this is a request every team can decline.

Thank you

Any concerns or questions?

smartagents.be